

Fabric Flo — Complete Roadmap (Mac → Windows → Both App Stores)

Your step-by-step path from this Mac to a live product on iPhone and Android.

Michael Keating · June 2026 · Plain English · Not legal advice

GitHub repo: `https://github.com/KeatingMichael/fabric-flo`

Live site (when deployed): `https://fabricflo-app.com`

App ID (both stores): `app.fabricflo.tracker`

How to use this document

- Work **top to bottom**. One part at a time.
- Check boxes as you finish each step.
- You do **not** need to understand the code — copy/paste values where indicated.
- **Print or save the PDF:**
`docs/Fabric_Flo_Complete_Roadmap.pdf` (same content).

Other PDFs (optional detail):

PDF	When to open it
<code>Fabric_Flo_Transfer_To_Windows.pdf</code>	Extra detail on cloning on Windows
<code>Fabric_Flo_Codemagic_Setup.pdf</code>	Fill-in-the-blanks for Apple/Google keys
<code>Fabric_Flo_Finish_And_Submit_Checklist.pdf</code>	Store submission tick-list
<code>Fabric_Flo_START_HERE.pdf</code>	Short overview if you feel lost

Part 0 — Before you leave this Mac (30 minutes)

Do this **once** while you still have the project open on the Mac.

0.1 Save secrets somewhere safe (password manager)

`.env` is **not** on GitHub. On Windows you will recreate it. Write down these values now:

Variable	Where to find it
<code>VITE_SUPABASE_URL</code>	Supabase → Project Settings → API
<code>VITE_SUPABASE_ANON_KEY</code>	Same page (anon public key)
<code>VITE_PUBLIC_APP_URL</code>	Your live URL, e.g. <code>https://fabricflo-app.com</code> (no trailing slash)
<code>VITE_FABRIC_FLO_BACKEND</code>	Always normalized
<code>VITE_SUPPORT_EMAIL</code>	Your support email
<code>VITE_PRIVACY_EMAIL</code>	Your privacy email

Supabase Edge secrets (operator only — Supabase dashboard, not `.env`):

Secret	Purpose
<code>GEMINI_API_KEY</code>	Handwritten label OCR (primary)
<code>GOOGLE_VISION_API_KEY</code>	Optional OCR fallback
<code>OCR_SPACE_API_KEY</code>	Optional free OCR fallback

Project ref: `zfrekjlqpkipuoliptpd`

0.2 Push latest code to GitHub

On the Mac, in the project folder:

```
git status
git push origin main
```

If `git push` succeeds, Windows will download the latest version. You do **not** need to copy the folder on a USB stick.

0.3 Confirm the website still builds (optional)

```
npm run build
```

If it finishes without errors, you're good to hand off the Mac.

Part 1 — Move to Windows 11 (1–2 hours)

1.1 Install tools on Windows

Download and install (defaults are fine):

Tool	Link
Git for Windows	https://git-scm.com/download/win
Node.js 20 LTS	https://nodejs.org/en/download
GitHub Desktop <i>(easiest)</i>	https://desktop.github.com
Cursor or VS Code	https://cursor.com
Android Studio <i>(for Google Play — can wait until Part 5)</i>	https://developer.android.com/studio

Open **PowerShell** or **Git Bash** and verify:

```
node -v    # should print v20.x
git --version
```

1.2 Clone the project from GitHub

Option A — GitHub Desktop (recommended):

1. File → Clone repository
2. Sign in as **KeatingMichael**
3. Select **fabric-flo**
4. Clone to e.g. **C:\Users\You\Documents\fabric-flo**

Option B — Command line:

```
cd ~/Documents
git clone https://github.com/KeatingMichael/fabric-flo.git
cd fabric-flo
```

1.3 Install and run locally

```
npm install
copy .env.example .env      # PowerShell: Copy-Item
                              .env.example .env
```

Edit `.env` — paste the values you saved in Part 0.1.

```
npm run dev
```

Open `http://localhost:5173` → marketing page loads

Open `http://localhost:5173/app` → app loads

Transfer complete when both URLs work.

1.4 Sanity check on Windows

- ☐ `node -v` is v20.x
- ☐ Project cloned from GitHub
- ☐ `npm install` finished
- ☐ `.env` recreated with Supabase + URL values
- ☐ `npm run dev` works
- ☐ Test edit → `git push` succeeds

Part 2 — Live backend + website (1 evening)

Required **before** App Store or Google Play submission (privacy URL, sign-in, password reset).

2.1 Supabase (database + login)

- ☐ Production project exists (`zfrekjlpkipuoliptpd` or your project)
- ☐ Run migrations: Supabase SQL Editor → paste contents of `supabase/APPLY_ALL_MIGRATIONS.sql`
(Regenerate first on Windows: `npm run supabase:bundle`)
- ☐ **Authentication → URL configuration:** Site URL and redirect URLs match `VITE_PUBLIC_APP_URL`
- ☐ Edge function deployed: `supabase functions deploy label-ocr --project-ref YOUR_REF`
- ☐ Edge secret set: `GEMINI_API_KEY` (Google Cloud → Generative Language API enabled)

See `docs/BACKEND_SETUP.md` and `docs/LABEL_OCR_SETUP.md` for detail.

2.2 Netlify (public website + PWA)

- ☐ Account at <https://app.netlify.com>
- ☐ Connect GitHub repo `KeatingMichael/fabric-flo`
- ☐ Set **environment variables** (same as `.env`):
 - `VITE_SUPABASE_URL` , `VITE_SUPABASE_ANON_KEY`
 - `VITE_FABRIC_FLO_BACKEND=normalized`
 - `VITE_PUBLIC_APP_URL` , `VITE_SUPPORT_EMAIL` , `VITE_PRIVACY_EMAIL`
- ☐ Deploy → confirm these URLs load:
 - `/` (marketing)
 - `/app` (productions home)
 - `/privacy`
 - `/terms`

2.3 Release check

```
npm run verify:release
```

Fix anything it flags before store submission.

Part 3 — Store developer accounts (do early — approvals take time)

Store	Cost	Sign up
Apple Developer Program	\$99/year	https://developer.apple.com
Google Play Console	\$25 one-time	https://play.google.com/console

- ☐ Apple enrollment approved
 - ☐ Google Play account created
 - ☐ Listing copy drafted — see **docs/STORE_LISTING.md**
 - ☐ Demo login for app reviewers — see **docs/APP_REVIEW_NOTES.md**
-

Part 4 — Who uses what (your product plan)

User	Typical device	Label scan quality
Crew	Phone app (install from store)	Best — Apple Vision on iPhone (native app)
Dept heads	Phone or browser	Log/export; scan less often
Coordinator	Phone or laptop	Invite Code, setup

Important: Label scanning in **Safari** (fabricflo-app.com) uses **cloud OCR** (slower). **TestFlight / App Store build** on iPhone uses **on-device Vision first** — that is the scan experience crew should get.

Part 5 — Android app (Windows + Android Studio)

All on your Windows 11 PC.

```
npm run build
npx cap sync android
```

```
npm run cap:android # opens Android Studio
```

In Android Studio:

- ☐ **Build → Generate Signed App Bundle (.aab)**
- ☐ Create **upload keystore** — back up the `.jks` file and passwords forever
- ☐ Confirm `versionCode 1 / versionName "1.0.0"` in `android/app/build.gradle`

Google Play Console:

- ☐ Create app → **Internal testing** track → upload `.aab`
- ☐ Data safety form, content rating, privacy URL (`https://YOUR_DOMAIN/privacy`)
- ☐ Screenshots + feature graphic
- ☐ Install on Android phone from internal testing → test QR scan + label scan + sync
- ☐ Promote to **Production** when ready (staged rollout recommended)

Part 6 — iOS app (Codemagic — no Mac required)

Windows **cannot** build iOS locally. Use **Codemagic** (cloud Mac).

6.1 Codemagic setup

- ☐ Sign up <https://codemagic.io> → **Sign in with GitHub** → authorize **fabric-flo**
- ☐ Fill in **Fabric_Flo_Codemagic_Setup.pdf** (every blank field)
- ☐ App Store Connect → create app with bundle ID **app.fabricflo.tracker**
- ☐ Upload Apple API key (`.p8`) to Codemagic integration **FabricFloAppStoreKey**
- ☐ Set Codemagic env vars (`VITE_*` + Apple keys per `codemagic.yaml`)

6.2 Build and TestFlight

- ☐ Run Codemagic workflow `ios-release`
- ☐ Wait for build → appears in **TestFlight**
- ☐ Install **TestFlight** on your iPhone → open Fabric Flo beta

6.3 Test on a real iPhone (this is the scan test that matters)

With TestFlight build installed (not Safari):

- ☐ Sign in / create production
- ☐ Add a **place** (Filming loc → name → Add)
- ☐ Scan tab → **Rental label**
- ☐ Frame label `111023 / SOLID / 12' x 12'` → **Tap Scan**
- ☐ Fields fill in a few seconds → Add to Log
- ☐ Scan a **dynamic QR** on a fabric piece
- ☐ Airplane mode → scan → reconnect → sync completes

If labels fail in TestFlight but work in nothing else, check Supabase edge logs for Gemini errors.

6.4 Submit to App Store

- ☐ App Store Connect: screenshots, description, App Privacy labels, age rating 4+
 - ☐ Privacy Policy URL = `https://YOUR_DOMAIN/privacy`
 - ☐ Paste reviewer notes from `docs/APP_REVIEW_NOTES.md`
 - ☐ **Submit for Review** (usually 1–3 days)
-

Part 7 — On-set testing before public launch

- ☐ TestFlight (iOS) + Play internal testing (Android) with real crew
 - ☐ Offline: scan → place → log → sync when back online
 - ☐ Rental list export (CSV + PDF) from a phone
 - ☐ Invite Code flow: coordinator creates code → crew joins → shared log
 - ☐ Account deletion + forgot-password email
-

Part 8 — Go live and operate

- ☐ Apple approved → App Store live
- ☐ Google promoted → Play Store live
- ☐ Monitor Supabase usage and errors
- ☐ For updates: fix code → bump version → `git push` → rebuild
(Codemagic + Android Studio) → resubmit

Typical update commands on Windows:

```
git pull
npm install
npm run build
npx cap sync android          # Android
npm run netlify:deploy        # website (if you use CLI
deploy)
# iOS: trigger ios-release in Codemagic
```

Quick reference — commands you'll use most

```
npm install          # first time on a machine
npm run dev          # run locally in browser
npm run build        # production web build
npm run verify:release # pre-release checks
npm run guides:pdf   # regenerate all guide PDFs
npx cap sync android # push web build into Android
shell
npm run cap:android  # open Android Studio
```

iOS builds: Codemagic dashboard → `ios-release` → Run (not on Windows locally).

Costs to launch (rough)

Item	Cost
------	------

Google Play	\$25 once
Apple Developer	\$99/year
Supabase + Netlify free tiers	\$0 to start
Codemagic	Free tier often enough for early builds
Total to first launch	~\$124 plus optional freelancer for Codemagic setup (~\$50–150)

If you get stuck

Problem	Where to look
Windows clone / <code>.env</code>	<code>docs/TRANSFER_TO_WINDOWS.md</code>
Supabase / migrations	<code>docs/BACKEND_SETUP.md</code>
Label OCR / Gemini	<code>docs/LABEL_OCR_SETUP.md</code>
Codemagic keys	<code>docs/CODEMAGIC_SETUP.md</code>
Store listing text	<code>docs/STORE_LISTING.md</code>
App reviewer demo login	<code>docs/APP_REVIEW_NOTES.md</code>
Legal (not advice)	<code>npm run legal:pdf</code>

Fabric Flo — Film fabric & bag tracker for productions.

Repo: <https://github.com/KeatingMichael/fabric-flo>